

Subject: Artificial Intelligence

Date: 26/09/2025

Exam: AY_25-26_E3SEM1_Monthly Test-II

Subject Code: CS3103

Time: 45 Mins.

Max Marks: 15

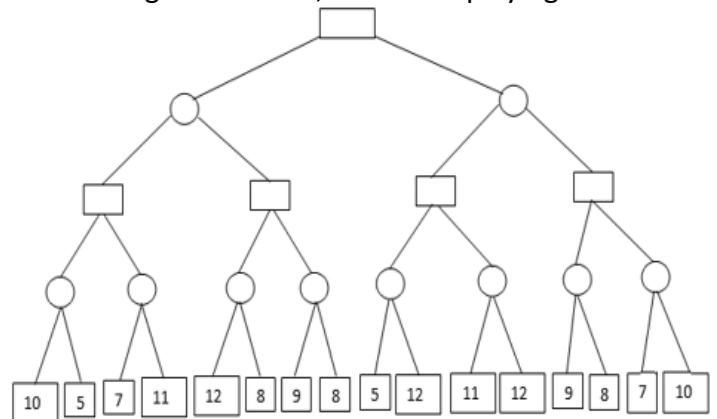
Section-A [5-Marks]

- Which of the following sentences can be expressed as Horn clause form?
S1: $((\neg P \vee \neg Q) \rightarrow R) \wedge (\neg S \rightarrow R)$
S2: $((\neg P \vee \neg Q) \rightarrow \neg R) \wedge (S \rightarrow R)$
- Which of the sentences in the previous question can be expressed in CNF form?
- An evaluation function is needed in a game like Chess because:
 - Even though the game tree can be fully explored we can still harness human knowledge
 - The game tree is too big to be fully explored and we need to evaluate non-leaf nodes
 - To be able to generate an explanation for the move played by the computer
 - None of the above
- Completeness guarantees that there are no false statements within a logical system. True or False?
- Soundness means that every provable statement in a logical system is true. True or False?

Section B: Answer Any Two [10-Marks]

1. Using resolution refutation, prove that the following goal can be derived from the premises.
 - a. If Henry has two days off, then if the weather is bad, Henry is not fishing.
 - b. If Henry is not fishing and is not playing cards with his friends, then he is watching TV at home.
 - c. If Henry is working, then he is neither playing cards with his friends nor watching TV at home.
 - d. Henry has two days off.

Goal: If the weather is bad and Henry is not watching TV at home, then he is playing cards with his friends.



2. Apply alpha-beta pruning on the following game tree. The rectangles represent Max nodes and the circles represent Min nodes. How many nodes will be pruned by alpha-beta search?
3. Explain AND-OR Graph. Write a program for forward chaining. [2 + 3 Marks]

Figure 2

Answer Keys

Section - A

S1: $((\neg P \vee \neg Q) \rightarrow R) \wedge (\neg S \rightarrow R)$
CNF: $(P \vee R) \wedge (Q \vee R) \wedge (S \vee R)$

S2 $((\neg P \vee \neg Q) \rightarrow \neg R) \wedge (S \rightarrow R)$
CNF: $(\neg R \vee P) \wedge (\neg R \vee Q) \wedge (\neg S \vee R)$

1. All the clauses in S2 are in Horn clause form.
2. Both can be expressed in CNF form.
3. An evaluation function is needed in a game like Chess because:
 - b. The game tree is too big to be fully explored and we need to evaluate non-leaf nodes

Soundness $\rightarrow$ no false derivation

Completeness $\rightarrow$ all true things are provable

4. Completeness guarantees that there are no false statements within a logical system. $\Rightarrow$ False
5. Soundness means that every provable statement in a logical system is true. $\Rightarrow$ True

Section - B

1.

Symbols:

T = Henry has two days off

B = Weather is bad

F = Henry is fishing

C = Henry is playing cards with friends

W = Henry is watching TV at home

K = Henry is working

Premises (original form):

P1: $T \rightarrow (B \rightarrow \neg F) = \neg T \vee \neg B \vee \neg F$

P2: $(\neg F \wedge \neg C) \rightarrow W = F \vee C \vee W$

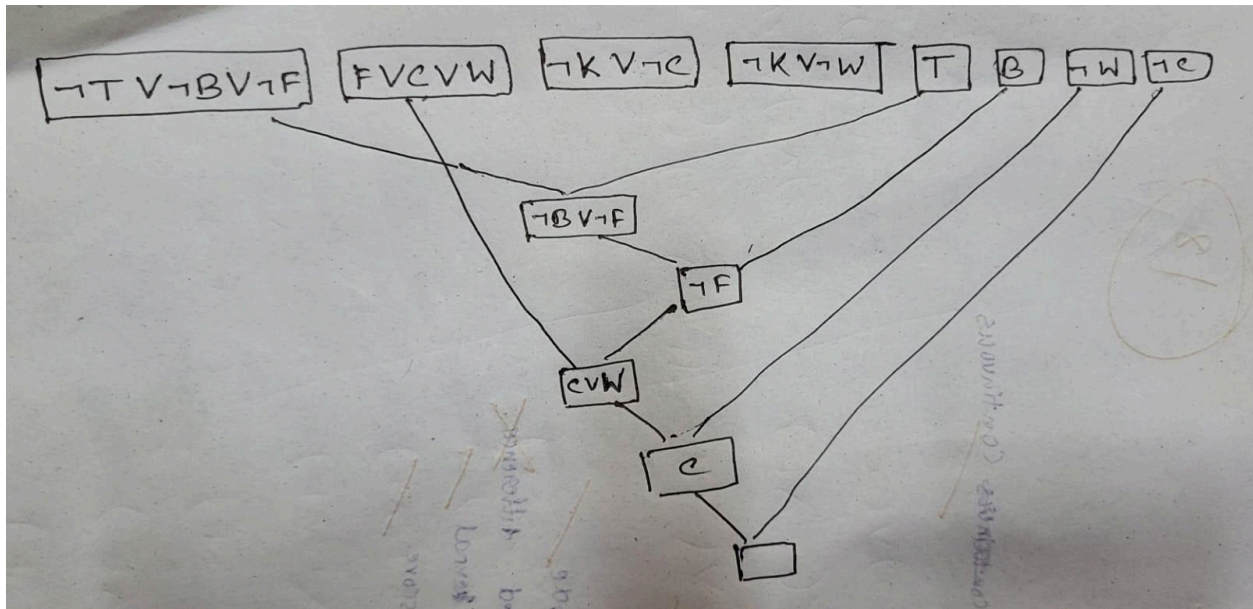
P3: $K \rightarrow (\neg C \wedge \neg W) = (\neg K \vee \neg C) \wedge (\neg K \vee \neg W)$

P4: T

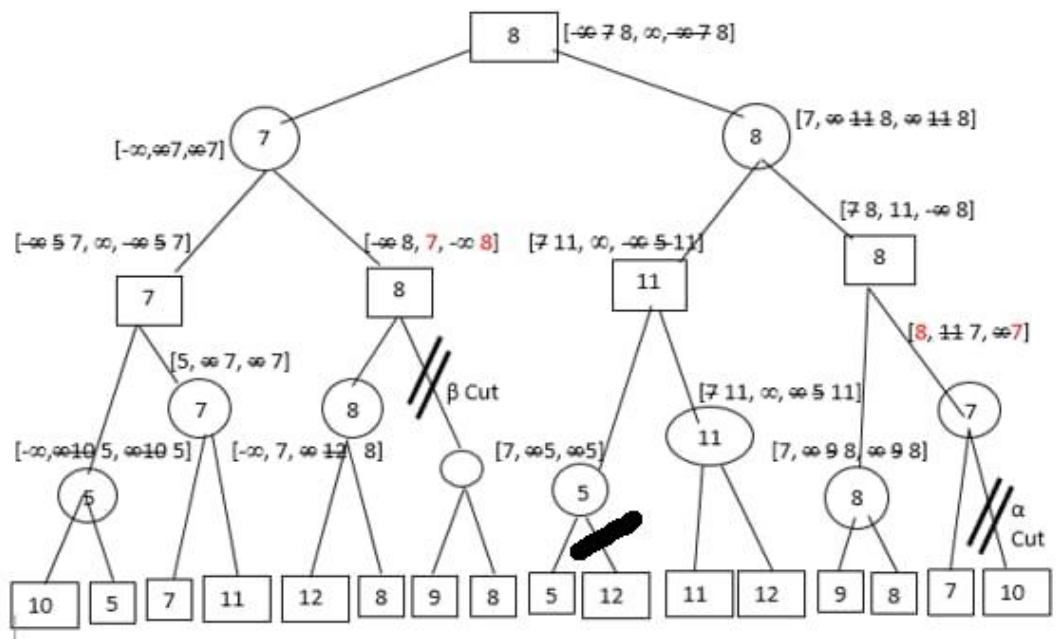
Goal: $(B \wedge \neg W) \rightarrow C$

Negated goal (for refutation) in CNF:

$$\neg G: \neg((B \wedge \neg W) \rightarrow C) = B \wedge \neg W \wedge \neg C$$



2.



Pruned internal count: 1

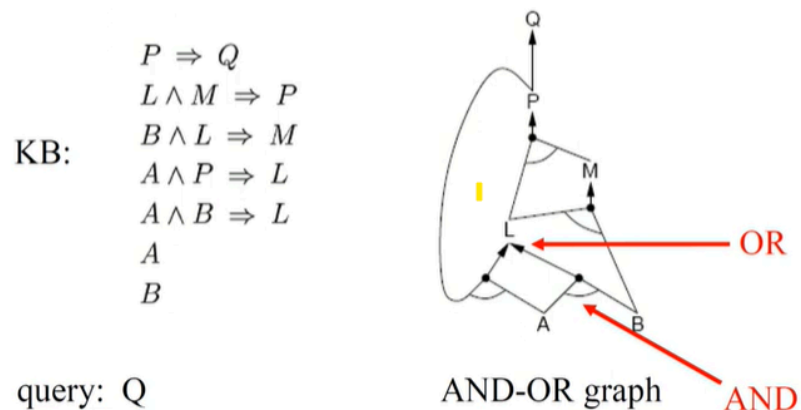
Pruned leaves count: 4 values (left→right): [9, 8, 12, 10]

3. An AND-OR graph represents Horn rules where OR-nodes are conclusions (a fact can be derived by any applicable rule) and AND-nodes are rule bodies (all

premises must be true). It visually supports forward chaining by showing how facts propagate until the query is derived

Forward chaining

- Fire any rule whose premises are satisfied in the KB
- Add its conclusion to the KB until query is found



```
def forward_chaining(facts, rules, goal=None):
    known = set(facts)
    changed = True
    while changed:
        changed = False
        for premises, conclusion in rules:
            if conclusion not in known and
                premises.issubset(known):
                known.add(conclusion)
                changed = True
            if goal and conclusion == goal:
                return known, True
    return known, (goal in known if goal else None)
```

```
#Additional code
# Knowledge Base
facts = {"A", "E"}
rules = [
    ({ "A" }, { "B" },          # A → B
    ({ "E" }, { "C" },          # E → C
    ({ "B", "C" }, { "D" },     # B ∧ C → D
]
```

```
# Run forward chaining
known, proved = forward_chaining(facts, rules, goal="D")

print("Known facts:", known)
print("Goal D proved?", proved)
```